

WS63 OHOS GPIO 使用方法

本文档说明 WS63 OHOS 下 GPIO 的原理、开发板引脚、LED 模块连线、gpio_led 示例及编译烧录方法。

适用平台：九联星闪开发板，基于海思 WS63E 芯片。

文档更新于 2026-03-12，为本人在开发中总结的经验，如有问题请及时联系王老师 (wangpeizheng@sztu.edu.cn)。

一、GPIO 原理

1.1 什么是 GPIO

GPIO (General Purpose Input/Output, 通用输入/输出) 是微控制器上可编程的引脚，既可配置为输入也可配置为输出，用于与外部设备进行数字信号交互。

1.2 输入与输出

模式	说明
输出	引脚由程序控制输出高电平 (通常 3.3V) 或低电平 (0V)，用于驱动 LED、继电器等
输入	引脚读取外部电平状态，用于检测按键、传感器等

1.3 推挽与开漏

- 推挽输出：**可主动输出高/低电平，驱动能力强，常用于 LED 控制。
- 开漏输出：**只能拉低或高阻，需外接上拉电阻，常用于总线共享。

1.4 电平逻辑

- 高电平：**通常为 3.3V (或芯片工作电压)，逻辑 1。
- 低电平：**通常为 0V，逻辑 0。
- LED 驱动：**常见两种接法——**低电平点亮** (阳极接 VCC，阴极经限流电阻接 GPIO) 或 **高电平点亮** (阳极经限流电阻接 GPIO，阴极接 GND)。

二、九联星闪开发板引脚

2.1 开发板概述

九联星闪开发板基于海思 WS63E 芯片，支持星闪、Wi-Fi、BLE 等通信，提供丰富的 GPIO 及外设接口。

2.2 GPIO 资源

WS63 芯片具有多组 GPIO，通过引脚复用支持：

- **GPIO**：通用输入输出
- **SPI** × 1、**QSPI** × 1
- **I2C** × 2、**I2S** × 1
- **UART** × 3、**ADC** × 6、**PWM** × 8

2.3 常用 GPIO 与 LED 对应关系

九联星闪开发板使用 **GPIO10** 通过排针与 LED 灯相连，为本示例默认配置。

GPIO 编号	说明	电平逻辑
GPIO 9	核心板板载 LED	输出低电平点亮
GPIO 10	排针引出，连接 LED（九联星闪开发板默认）	输出低电平点亮

详细引脚定义请参考《WS63 系列硬件用户指南》及开发板原理图。

2.4 排针与 LED 连接

九联星闪开发板通过排针将 GPIO10 引出，可直接连接 LED 模块或红绿灯扩展板。若使用扩展板，将扩展板排针插入核心板对应排母即可，红色 LED 对应 GPIO10。

三、开发板与 LED 模块连线

九联星闪开发板通过排针将 GPIO10 引出，可直接连接 LED 灯或扩展板。

3.1 使用排针 GPIO10 连接 LED（九联星闪开发板默认）

九联星闪开发板 GPIO10 通过排针引出，可直接连接 LED：

1. **自接 LED**：用杜邦线将排针上的 GPIO10 与 LED 相连，见下节。

3.2 自接 LED 模块

使用杜邦线连接外部 LED 模块时：

开发板	LED 模块	说明
排针 GPIO10	LED 阳极（经 220Ω ~ 1kΩ 限流电阻）	低电平点亮接法
GND	LED 阴极	共地

低电平点亮接法示意：

开发板排针 GPIO10 — [220Ω] — LED 阳极 — LED 阴极 — GND

限流电阻建议 220Ω ~ 1kΩ，避免电流过大损坏 LED 或 GPIO。

四、GPIO API 说明

4.1 常用 API

API	说明
<code>IoTGpioInit(id)</code>	初始化指定 GPIO
<code>IoTGpioSetDir(id, dir)</code>	设置方向: <code>IOT_GPIO_DIR_IN</code> / <code>IOT_GPIO_DIR_OUT</code>
<code>IoTGpioSetOutputVal(id, val)</code>	设置输出电平: <code>IOT_GPIO_VALUE0</code> (低) / <code>IOT_GPIO_VALUE1</code> (高)
<code>IoTGpioGetInputVal(id, val)</code>	读取输入电平
<code>IoTGpioDeinit(id)</code>	去初始化 GPIO

4.2 看门狗（防重启）

长循环任务需定期喂狗，否则芯片看门狗超时会触发 NMI 复位：

API	说明
<code>IoTWatchDogKick()</code>	喂狗，重置看门狗计时

头文件: `#include "iot_watchdog.h"`

4.3 头文件

```
#include "iot_gpio.h"
#include "iot_watchdog.h"  /* 长循环任务需喂狗时 */
```

五、gpio_led 示例项目

5.1 项目创建

```
python3 build/scripts/make_app.py -c gpio_led gpio_led_demo
```

5.2 项目结构

```
applications/sample/wifi-iot/app/gpio_led/
├─ gpio_led.c
└─ BUILD.gn
```

5.3 完整源码（此代码有bug，可能程序无法长时间运行，代码需要优化）

gpio_led.c

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */

#include <stdio.h>
#include "ohos_init.h"
#include "cmsis_os2.h"
#include "iot_gpio.h"
#include "iot_watchdog.h"

/* GPIO10： LED灯（低电平点亮） */
#define LED_GPIO 10
#define LED_TASK_STACK_SIZE 0x1000
#define LED_TASK_PRIO 25

static void GpioLedTask(void *arg)
{
    (void)arg;
    uint32_t count = 0;

    IoTGpioInit(LED_GPIO);
    IoTGpioSetDir(LED_GPIO, IOT_GPIO_DIR_OUT);

    while (1) {
        /* 低电平点亮 LED, 500ms */
        IoTGpioSetOutputVal(LED_GPIO, IOT_GPIO_VALUE0);
        printf("LED ON\r\n");
        osDelay(500);
        /* 高电平熄灭 LED, 500ms */
        IoTGpioSetOutputVal(LED_GPIO, IOT_GPIO_VALUE1);
        printf("LED OFF\r\n");
        osDelay(500);
    }
}

static void GpioLedEntry(void)
{
}
```

```

osThreadAttr_t attr = {
    "GpioLedTask", 0, NULL, 0, NULL, LED_TASK_STACK_SIZE, LED_TASK_PRIO, 0, 0
};

if (osThreadNew(GpioLedTask, NULL, &attr) == NULL) {
    printf("[GpioLed] Failed to create GpioLedTask!\r\n");
}
}

APP_FEATURE_INIT(GpioLedEntry);

```

BUILD.gn

```

static_library("gpio_led_demo") {
    sources = [ "gpio_led.c" ]

    include_dirs = [
        "//commonlibrary/utils_lite/include",
        "//kernel/liteos_m/components/cmsis/2.0",
        "//base/iothardware/peripheral/interfaces/inner_api",
    ]
}

```

5.4 使用核心板 LED (GPIO 10)

九联星闪开发板默认使用排针 GPIO10 连接 LED

六、编译

在源码根目录执行：

```
python3 build/scripts/make_app.py -b
```

或全量编译（清理后重编）：

```
python3 build/scripts/make_app.py -f
```

也可手动执行：

```
hb set -p nearlink_dk_3863
hb build -f
```

七、烧录

7.1 固件路径

```
out/nearlink_dk_3863/nearlink_dk_3863/ws63-liteos-app/ws63-liteos-app_all.fwpkg
```

7.2 烧录步骤

1. 连接开发板 USB，安装 CH340 驱动。
2. 打开 BurnTool，选择串口，芯片选择 **WS63**。
3. 选择 `ws63-liteos-app_all.fwpkg`，勾选 `Auto Burn`、`Auto disconnect`。
4. 点击 Connect，按 RST 进入烧录模式。
5. 烧录完成后按 Reset 重启。

7.3 WSL 环境

若源码在 WSL，Windows 可通过 `\\ws1.localhost\...\ws63-liteos-app_all.fwpkg` 访问固件路径。

八、运行结果

烧录并复位后：

- **九联星内开发板**：GPIO10 通过排针连接的 LED 每 500ms 闪烁一次（若使用红绿灯扩展板，则为黄色 LED）。
- **串口**：波特率 115200，任务创建失败时会打印错误信息；正常运行无额外输出（已去掉循环内 `printf`，避免串口阻塞影响喂狗）。

九、常见问题

9.1 LED 闪烁时间与预期不符

WS63 系统 tick 为 **100Hz**（1 tick = 10ms），`osDelay(n)` 的参数是 **tick 数**而非毫秒。若误写 `osDelay(500)`，实际延时为 $500 \times 10\text{ms} = 5\text{秒}$ 。正确做法：500ms 应使用 `osDelay(50)`，或通过 `MS_TO_TICKS(500)` 宏转换。

9.2 运行一段时间后系统重启

原因：芯片看门狗超时。WS63 默认启用看门狗（约 8 秒超时），若长时间未喂狗会触发 NMI 复位（日志中可见 `mcause: 0x8000000c`、`NON_OS_NMI_CWDT_INT_FLAG`）。此外，`printf` 过于频繁时串口缓冲满会阻塞，影响喂狗时机。

解决方案：

1. **在长循环中定期喂狗**：引入 `iot_watchdog.h`，在循环内调用 `IoWatchDogKick()`。本示例每 2 次闪烁（约 2 秒）喂狗一次，确保在 8 秒超时前完成。
2. **减少或去掉循环内 printf**：避免串口阻塞影响喂狗。

十、参考文档

- [03-环境配置和系统编译烧录文档.md](#)：环境配置、编译、烧录
- [04-创建新项目的方法.md](#)：创建新示例项目
- 星闪技术资料：https://gitee.com/hihopeorg_group/near-link

- WS63 硬件用户指南: <https://hispark-obs.obs.cn-east-3.myhuaweicloud.com/>
- 常见问题见本文档第九章